手写数字识别

1. 实验目的

1.1 了解 MNIST 数据集；

1.2 掌握 LeNet-5 网络的结构；

1.3 完成手写数字识别任务。

2. 实验环境

SUB KIT NX 嵌入式开发板。

3. 实验数据

MNIST 数据集；

4. 导入数据集

torchvision 模块中含有 MNIST 数据集，运行如下代码，可以自行下载，也可以提前下载好 MNIST 数据集，更改路径即可。

· 加载训练数据，是 Tensor 类型

```
train_loader = torch.utils.data.DataLoader(
    datasets.MNIST(root='./data', train=True, download=True,
                   transform=transforms.Compose([
                       transforms.ToTensor(),
                       transforms.Normalize((0.1307,), (0.3081,)) ])) # 数据集给出的均值和标准差系数，每个数据集都不同
                   batch_size=batch_size, shuffle=True)
```

· 加载测试数据集，是 Tensor 类型

```
test_loader = torch.utils.data.DataLoader(
datasets.MNIST(root='./data',train=False,transform=transforms.
Compose([    transforms.ToTensor(),
             transforms.Normalize((0.1307,), (0.3081,))
        ])), batch_size=test_batch_size, shuffle=True)
```

root 参数：下载地址

train 参数：要载入的是否是训练集，True 是训练集，False 不是

transform 参数：下载格式

dataset 参数：要装载的训练集

batch_size 参数：一批中有多少张图像

shuffle 参数：是否打乱图像顺序

5. 模型构建

本实验采用的是 LeNet-5 网络架构，结构如图 1 所示。选择 ReLU 函数作为激活函数。搭建过程如下：

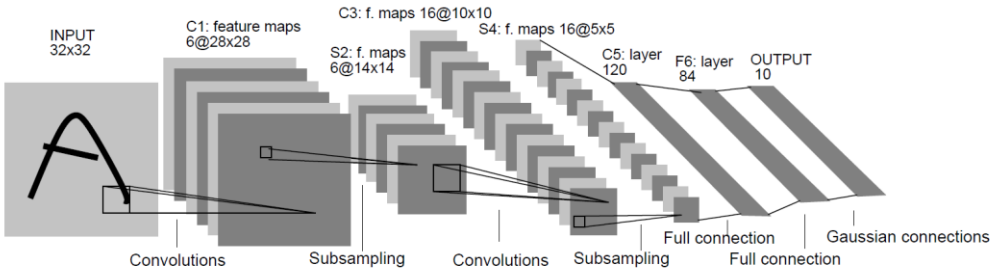

图 1　LeNet-5 网络结构

```
class LeNet(nn.Module):                          #定义类，存储网络结构
    def __init__(self):
        super(LeNet, self).__init__()
        self.conv1 = nn.Sequential(        # input_size=(1*28*28)
            nn.Conv2d(1, 6, 5, 1, 2),      # padding=2 保证输入输出尺寸相同
            nn.ReLU(),                     # ReLU 激活函数，input_size=(6*28*28)
            nn.MaxPool2d(kernel_size=2, stride=2),    # output_size=(6*14*14)
        )
        self.conv2 = nn.Sequential(
            nn.Conv2d(6, 16, 5),
            nn.ReLU(),                     # input_size=(16*10*10)
            nn.MaxPool2d(2, 2)             # output_size=(16*5*5)
        )
          ......
    def forward(self, x):                  #  定义前向传播过程，输入为 x
```

```
        x = self.conv1(x)
        x = self.conv2(x)
# nn.Linear()的输入输出维度为一维，所以要把多维度 tensor 展平成一维
        x = x.view(x.size()[0], -1)
        ……
        return x                                # 返回值
```

5.1 训练网络

构建网络以后，便可以对网络进行训练了，采用 SGD 算法进行优化，采用交叉熵作为损失函数，代码如下：

```
def train(epoch):
    model.train()                   # 设置为训练模式
    for batch_idx, (data, target) in enumerate(train_loader):
        data = torch.tensor(data).type(torch.FloatTensor).cuda()
        target = torch.tensor(target).type(torch.LongTensor).cuda()
        torch.optim.SGD(model.parameters(), lr=lr,
momentum=momentum).zero_grad()
        output = model(data)     # 把数据输入网络并得到输出（前向传播）
        loss = nn.CrossEntropyLoss()(output, target)
        loss.backward()
        torch.optim.SGD(model.parameters(), lr=lr, momentum=momentum).step()
#更新参数
        if batch_idx % log_interval == 0:   # 打印相关信息
            print('Train Epoch: {} [{}/{} ({:.0f}%)]\tLoss: {:.6f}'.format(
                epoch, batch_idx * len(data), len(train_loader.dataset),
                        100. * batch_idx / len(train_loader), loss.item()))
```

5.2 测试模型

为了测试该网络的准确率，需要构建测试模型，代码如下：

```
def test():
    model.eval()          # 设置为测试模式
    test_loss = 0         # 初始化测试损失值为 0
```

```
    correct = 0              # 初始化预测正确的数据个数为 0
    for data, target in test_loader:
        data = torch.tensor(data).type(torch.FloatTensor).cuda()
        target = torch.tensor(target).type(torch.LongTensor).cuda()
        output = model(data)
        ……# 把所有 loss 值进行累加
        ……# 获得最大概率的下标
        ……# 对预测正确的数据个数进行累加
        ……# 所有 loss 值进行过累加，除以总的数据长度得平均 loss
    print('\nTest set: Average loss: {:.4f}, Accuracy: {}/{} ({:.0f}%)\n'.format(
        test_loss, correct, len(test_loader.dataset), 100. * correct /
len(test_loader.dataset)))
```

设置的参数；

```
lr = 0.01                # 学习率
momentum = 0.5
log_interval = 10        # 跑多少次 batch 进行一次日志记录
epochs = 10
batch_size = 64
test_batch_size = 1000
```

6. 实验任务

基于 LeNet-5 网络实现 MNISIT 数据集的数字识别任务。